ASSIGNMENT 2

Student Name: Thikkavarapu Sanhith

Register Number: 192311228

Submission Date: 05-Nov-2026

Course: CSA4704 – Deep Learning

Q1 – Hidden units: activation function and depth (6 marks)

What this question is testing

The question asks whether the activation function changes how easily a deep network can learn. A hidden layer first computes a weighted sum z, then applies an activation φ(z). During backpropagation, the error signal is passed from the output layer towards the input. Each layer multiplies that signal by its weight matrix and by the derivative of its activation. If these repeated multiplications make the signal extremely small, the early layers barely update their weights. This is called the vanishing-gradient problem.

1. Mathematical prediction before implementation

For hidden layer l, the layer first computes an affine transformation and then applies the activation. During backpropagation, the error signal is multiplied by the next layer weights and by the activation derivative. The weight gradient is formed from the previous activation and the current error signal.

$$z_l = W_l a_{l-1} + b_l \quad \text{(Q1.1)}$$

$$a_l = \varphi(z_l) \quad \text{(Q1.2)}$$

$$\delta_l = (W_{l+1}^T \delta_{l+1}) \odot \varphi'(z_l) \quad \text{(Q1.3)}$$

$$\partial L / \partial W_l = (1/m)\, a_{l-1}^T \delta_l \quad \text{(Q1.4)}$$

Sigmoid

The sigmoid maps any real input into the interval (0, 1). Its derivative is never greater than 0.25 and approaches zero for strongly positive or negative inputs. Repeated use across many layers can therefore attenuate the backward signal.

$$\sigma(z) = 1 / (1 + e\text{^}(-z)) \quad \text{(Q1.5)}$$

$$\sigma'(z) = \sigma(z)\,[1 - \sigma(z)] \quad \text{(Q1.6)}$$

$$\max \sigma'(z) = 0.25 \quad \text{(Q1.7)}$$

$$0.25^8 = 1.5259 \times 10^{-5} \quad \text{(Q1.8)}$$

Tanh

Tanh produces zero-centred outputs in (−1, 1). Around zero its derivative is close to one, which can help gradients pass through. For large absolute inputs, tanh saturates and its derivative approaches zero.

$$\tanh(z) = (e^z - e^{-z}) / (e^z + e^{-z}) \quad \text{(Q1.9)}$$

$$\tanh'(z) = 1 - \tanh^2(z) \quad \text{(Q1.10)}$$

$$\max \tanh'(z) = 1 \quad \text{(Q1.11)}$$

ReLU

ReLU passes positive inputs unchanged and maps non-positive inputs to zero. Its derivative is one on the active side and zero on the inactive side. It avoids derivative shrinkage for active neurons, but a neuron that remains inactive for all training examples receives no gradient.

$$\mathrm{ReLU}(z) = \max(0, z) \quad \text{(Q1.12)}$$

$$\mathrm{ReLU}'(z) = 1, \text{ if } z > 0;\ 0, \text{ if } z \leq 0 \quad \text{(Q1.13)}$$

Activation	Derivative	Expected behaviour as depth grows	Main risk
Sigmoid	$\sigma(z)[1 - \sigma(z)] \leq 0.25$	Repeated derivatives tend to shrink early-layer gradients.	Saturation and vanishing gradients.
Tanh	$1 - \tanh^2(z) \leq 1$	Can pass gradients well near $z = 0$; usually better centred than sigmoid.	Saturation when $\lvert z \rvert$ is large.
ReLU	1 if $z > 0$; otherwise 0	Active paths pass gradients without derivative shrinkage.	Dead units if their inputs remain non-positive.

Initialisation formulas

The initialisation controls the scale of weights before training. He initialisation is designed for ReLU-like activations. Xavier/Glorot initialisation is a common choice for tanh and sigmoid. Biases are set to zero in this experiment.

$$\mathrm{Var}(W) = 2 / \mathrm{fan_in} \quad \text{(He / Kaiming)} \quad \text{(Q1.14)}$$

$$\mathrm{Var}(W) = 2 / (\mathrm{fan_in} + \mathrm{fan_out}) \quad \text{(Xavier / Glorot)} \quad \text{(Q1.15)}$$

2. Three deliberately designed configurations

Use the same 30 input features, 64 units per hidden layer, one sigmoid output and binary cross-entropy loss. The shallow model is the reference; the deep model directly tests the bank requirement of at least eight hidden layers; the edge cases keep the deep ReLU architecture but deliberately use an unsuitable weight scale.

Configuration	Architecture / hidden units	Initialisation	Reason for including it
Shallow reference	2 hidden layers × 64 units; ReLU	He/Kaiming normal	Shows the behaviour of a short network.
Deep – sigmoid	8 hidden layers × 64 units; sigmoid	Xavier/Glorot	Tests repeated derivatives bounded by 0.25.
Deep – tanh	8 hidden layers × 64 units; tanh	Xavier/Glorot	Tests whether centred activations retain gradients.
Deep – ReLU	8 hidden layers × 64 units; ReLU	He/Kaiming normal	Tests active-path gradient flow in the required depth.
Edge – small weights	8 hidden layers × 64 units; ReLU	Normal weights, $\sigma = 0.01$	Tests whether very small weights make gradients tiny.
Edge – large weights	8 hidden layers × 64 units; ReLU	Normal weights, $\sigma = 2.0$	Tests unstable activation scale and possible output saturation.

3. Dataset, controls and measurements

Dataset used for this experiment: the public Wisconsin Breast Cancer Diagnostic dataset available through scikit-learn. It contains 569 rows and 30 numeric features. The stratified split is 341 training rows, 114 validation rows and 114 test rows (60% / 20% / 20%). Standardisation is fitted only on the training portion and then applied to validation and test data. This is a public tabular binary-classification proxy; it is not a credit-card fraud dataset, so its scores must not be presented as fraud-detection performance. In scikit-learn this dataset uses label 1 for benign and label 0 for malignant; therefore the reported precision and recall use benign as the positive class. A real bank experiment must encode fraud as the positive class and calculate fraud precision and recall.

Keep the data split, 64-unit width, optimiser, learning rate, batch size and epoch budget the same across comparisons. The experiment uses Adam with learning rate 0.001, batch size 32 and 15 epochs, and repeats the run with random seeds 42, 43 and 44. Sigmoid and tanh hidden layers use Xavier initialisation; ReLU hidden layers use He initialisation. The edge cases replace the hidden-layer weight initialisation with a normal distribution of standard deviation 0.01 or 2.0. The output layer remains a sigmoid unit.

Measure (1) the weight-gradient norm $G_l = \|\partial L/\partial W_l\|_F$ for each hidden layer on the first mini-batch of epoch 1, before the first optimiser update (the Frobenius norm is the L2 norm of all entries in the layer weight-gradient matrix); (2) the fraction of dead ReLU units after training, where a unit is counted as dead if its activation is non-positive for every training example; (3) final training binary cross-entropy; and (4) validation accuracy, precision and recall at a 0.5 threshold. Report the mean and sample standard deviation across the three seeds. The test split is reserved for final evaluation and is not used to choose the activation.

4. How to make the design decision

Do not select an activation using training loss alone. First check whether the earliest hidden layers receive useful gradients; then compare validation accuracy, precision and recall. Also check dead-ReLU fraction and run-to-run variation. The expected result is that sigmoid is most vulnerable to gradient shrinkage, tanh can be a good compromise when it remains unsaturated, and ReLU with He initialisation can preserve active-path gradients but may lose some units. The measured evidence and final recommendation are presented in the Q1 results subsection later in this report.

Q2 – Can You Trust Your Gradients? Backpropagation from First Principles (4 marks)

What this question is testing

This question checks whether the backpropagation equations are understood rather than treated as a framework feature. We build a network with one hidden sigmoid layer and one sigmoid output. The forward pass computes a prediction; the backward pass uses the chain rule to calculate how each weight and bias contributed to the loss. NumPy is used for the entire gradient-checking implementation. No automatic differentiation is used.

1. Computational graph and tensor shapes

The hospital context has four input measurements: age, length of stay, number of prior admissions and number of medications. For mathematical verification, the implementation uses a small batch with d = 4 input features and h = 5 hidden units. A separate training demonstration uses four standardised tabular proxy features from the public Breast Cancer Wisconsin dataset; this is not a hospital readmission dataset.

The forward graph is: input X → affine layer 1 → sigmoid hidden layer H → affine output layer → sigmoid probability P → binary cross-entropy loss. The computational graph figure in the evidence section shows the same data flow.

Symbol	Shape	Meaning
X	m × 4	Batch of m input records
W_1	4 × 5	Input-to-hidden weights
b_1	1 × 5	Hidden bias vector
H	m × 5	Hidden sigmoid activations
W_2	5 × 1	Hidden-to-output weights
b_2	1 × 1	Output bias
P, Y	m × 1	Predicted probabilities and binary labels

2. Forward propagation

First calculate each hidden pre-activation, apply sigmoid, then calculate the output logit and its probability. The same parameter matrices are shared by all examples in a mini-batch.

$$Z_1 = X W_1 + b_1 \quad \text{(Q2.1)}$$

$$H = \sigma(Z_1) \quad \text{(Q2.2)}$$

$$Z_2 = H W_2 + b_2 \quad \text{(Q2.3)}$$

$$P = \sigma(Z_2) \quad \text{(Q2.4)}$$

The binary cross-entropy loss averages the prediction error over the m examples. Probabilities are clipped away from exactly zero and one in the implementation so that log(0) is avoided numerically.

$$L = -(1/m) \Sigma_{i=1}^{m} [\, y_i \log(p_i) + (1-y_i) \log(1-p_i) \,] \quad \text{(Q2.5)}$$

3. Backpropagation derivation using the chain rule

Start at the output. For one sample, differentiate the loss with respect to the predicted probability. Then multiply by the sigmoid derivative. The two terms simplify to p − y. With mean loss over m examples, the batch error includes a factor 1/m.

$$\partial L / \partial p_i = -(1/m) [\, y_i/p_i - (1-y_i)/(1-p_i) \,] \quad \text{(Q2.6)}$$

$$\partial p_i / \partial z_{2i} = p_i (1-p_i) \quad \text{(Q2.7)}$$

$$\delta_2 = (P - Y) / m \quad \text{(Q2.8)}$$

The output weight gradient is the hidden activation matrix transposed multiplied by the output error. The output bias gradient is the sum of the output errors. These shapes follow directly from matrix multiplication.

$$\partial L / \partial W_2 = H^T \delta_2 \quad \text{(Q2.9)}$$

$$\partial L / \partial b_2 = \Sigma_i \delta_{2i} \quad \text{(Q2.10)}$$

Next propagate the error to the hidden layer. The gradient first passes through W_2 and then through the hidden sigmoid derivative $H \odot (1-H)$. Omitting this derivative is the deliberate bug used later in the verification experiment.

$$\partial L / \partial H = \delta_2 W_2^T \quad \text{(Q2.11)}$$

$$\delta_1 = (\delta_2 W_2^T) \odot H \odot (1-H) \quad \text{(Q2.12)}$$

$$\partial L / \partial W_1 = X^T \delta_1 \quad \text{(Q2.13)}$$

$$\partial L / \partial b_1 = \Sigma_i \delta_{1i} \quad \text{(Q2.14)}$$

Finally update every parameter using gradient descent. The learning rate η controls the step size. In this question the training demonstration uses full-batch gradient descent, while the gradient checker uses a fixed small batch.

$$\theta = \theta - \eta \nabla\theta L \quad \text{(Q2.15)}$$

4. Numerical gradient checking

Analytical gradients can contain an indexing, transpose or derivative error. Finite differences provide an independent numerical estimate for each parameter. The central-difference formula perturbs one parameter by $+\varepsilon$ and $-\varepsilon$, measures the change in loss, and divides by 2ε.

$$g_{num,i} = [L(\theta_i + \varepsilon) - L(\theta_i - \varepsilon)] / (2\varepsilon) \quad \text{(Q2.16)}$$

$$\text{Relative error} = \|g_{analytical} - g_{numerical}\|_2 / (\|g_{analytical}\|_2 + \|g_{numerical}\|_2 + 10^{-12}) \quad \text{(Q2.17)}$$

Use float64 and test two step sizes, $\varepsilon = 10^{-5}$ and $\varepsilon = 10^{-6}$. A correct implementation should have a very small relative error. For this well-conditioned small numerical fixture, the chosen acceptance threshold is 10^{-6}; errors below 10^{-8} provide a stronger margin. The threshold is not universal: very small gradients, poor scaling or floating-point precision can affect it.

ε	Parameter group	Correct backward error	Buggy backward error
10^{-5}	W_1	3.320×10^{-10}	0.6223
10^{-5}	b_1	2.297×10^{-10}	0.6413
10^{-5}	W_2	4.127×10^{-11}	4.127×10^{-11}
10^{-5}	b_2	3.704×10^{-12}	3.704×10^{-12}
10^{-6}	W_1	3.088×10^{-9}	0.6223
10^{-6}	b_1	9.022×10^{-9}	0.6413
10^{-6}	W_2	1.676×10^{-10}	1.676×10^{-10}
10^{-6}	b_2	3.826×10^{-10}	3.826×10^{-10}

5. Deliberate bug and interpretation

The deliberate bug removes $H \odot (1-H)$ from δ_1. The output layer gradients W_2 and b_2 remain correct because their calculation happens before the hidden-layer derivative. The W_1 and b_1 numerical errors become approximately 0.622 and 0.641, respectively. W_1 is the first parameter group in the backpropagation chain to expose the error; b_1 is also wrong because it is calculated from the same incorrect δ_1.

Training demonstration: four standardised input features, eight hidden sigmoid units, one sigmoid output, binary cross-entropy, full-batch gradient descent, $\eta = 0.05$ and 500 epochs. Across three seeds, the recorded test accuracy was 0.8567 ± 0.0051 and test BCE was 0.3468 ± 0.0145. These scores demonstrate that the implementation trains; gradient checking, not accuracy, is the evidence that the derivatives are correct.

Q3 – Engineer Overfitting, Then Control It: Regularisation (6 marks)

What this question is testing

This question asks you to deliberately create a generalisation problem, observe its signature in training and validation curves, and then test regularisation methods. A model that memorises its training examples may have very high training accuracy but substantially lower validation accuracy. The aim is to understand how capacity, data size and regularisation change this gap.

1. Experimental conditions and predictions

The public scikit-learn Digits dataset has 1,797 grayscale images of size 8×8 and ten digit classes. A stratified split provides 1,078 training images, 359 validation images and 360 test images. The generalisation condition uses 60% of the training pool (about 647 examples) and a 64-unit hidden layer. The overfit and regularisation conditions use 10% of the training pool (about 108 examples) and a larger MLP with hidden widths 128, 128 and 64. The additional condition corrupts 15% of the labels in that small training subset.

Condition	Data used	Network	Prediction before training
A – generalisation baseline	60% of training pool	64 hidden units	Smaller train–validation gap; model may have higher bias.
B – deliberate overfit	10% of training pool	128 → 128 → 64	Very high training accuracy, wider validation gap due to high capacity relative to data.
C – label noise	10% of training pool; 15% labels corrupted	128 → 128 → 64	Incorrect training targets make validation performance worse and increase uncertainty.

2. Bias–variance explanation

A model with too little capacity may fail to represent the pattern in both training and validation data. This is underfitting and is associated with higher bias. A model with excessive capacity relative to a small sample can fit idiosyncrasies of that sample; this is overfitting and is associated with higher variance. More training examples usually help constrain the set of functions consistent with the data. Corrupt labels make the learning target less reliable and can encourage memorisation.

The training–validation accuracy gap is calculated as follows:

$$\text{Accuracy gap} = \text{Accuracy_train} - \text{Accuracy_validation} \quad \text{(Q3.1)}$$

3. Regularisation methods and their mathematics

Method	How it changes training	Expected benefit and possible trade-off
L2 weight decay	Adds a penalty on squared weights.	Shrinks parameter magnitudes; excessive λ can underfit.
Dropout	Randomly masks hidden activations during training.	Reduces co-adaptation; too much dropout can make learning too difficult.
Early stopping	Stops when validation loss fails to improve for a patience window; restores best checkpoint.	Prevents late-stage memorisation; requires a validation set.
Data augmentation	Applies label-preserving transforms to training images.	Adds plausible variation; an unsafe transform can change the label.
Batch normalisation	Normalises mini-batch activations and learns scale/shift.	Can stabilise optimisation; it is not guaranteed to reduce overfitting.
Label smoothing	Moves hard class targets slightly toward a uniform distribution.	Discourages extreme confidence; too much smoothing can reduce class separation.

For L2 weight decay, the objective becomes:

$$L_{total} = L_{data} + (\lambda/2) \Sigma_l \|W_l\|^2_2 \quad \text{(Q3.2)}$$

For inverted dropout with keep probability $q = 1 - p$, the binary mask r is sampled during training and the retained activations are scaled so that their expectation remains unchanged:

$$\tilde{h} = (r \odot h) / q, \quad r_j \sim \text{Bernoulli}(q) \quad \text{(Q3.3)}$$

For multiclass label smoothing with smoothing amount α and K classes, the target is distributed between the original one-hot target and a uniform target:

$$y_smooth = (1-\alpha)y + (\alpha/K)\,1 \quad \text{(Q3.4)}$$

For data augmentation, $x' = T(x)$, where T is a transformation such as a small shift that preserves the digit class. Early stopping saves the weights at the epoch with the lowest validation loss, rather than necessarily using the final epoch.

4. What was compared and what the measurements show

The implementation compares L2 weight decay ($\lambda = 0.001$), dropout ($p = 0.5$), early stopping (patience 5), label smoothing ($\alpha = 0.1$), and a combined L2 + dropout condition. The base network, small-data subset, optimiser (Adam), learning rate, batch size and 60-epoch maximum are otherwise held constant. Each condition is repeated with seeds 42, 43 and 44.

Condition	Train acc.	Validation acc.	Gap	Test acc.	Best val. loss
More data / small model	0.996	0.956	0.040	0.957	0.142
Large model / 10% data	1.000	0.848	0.152	0.859	0.502
15% label noise	0.944	0.774	0.170	0.769	0.716
L2 weight decay	1.000	0.841	0.159	0.826	0.517
Dropout p = 0.5	0.897	0.769	0.128	0.763	0.680
Early stopping	0.994	0.850	0.144	0.831	0.515
Label smoothing 0.1	1.000	0.895	0.105	0.878	0.466
L2 + dropout	0.981	0.851	0.130	0.813	0.496

Interpretation: the small model with more data had the smallest average gap (about 0.040) and the highest test accuracy (about 0.957). The deliberately overfit model reached 1.000 training accuracy but only 0.848 validation accuracy. Label smoothing gave the best test accuracy among the small-data, high-capacity regularisation conditions (about 0.878) and reduced the gap to about 0.105. Dropout gave a smaller gap than the unregularised overfit baseline, but its train and validation accuracies were both lower, indicating that $p = 0.5$ may have been too strong for this short training budget. L2 + dropout reduced the gap but did not beat label smoothing on test accuracy.

The weight norm also decreased under L2 and L2 + dropout: approximately 12.73 and 12.47, compared with 14.91 for the unregularised overfit baseline. Lower weight norm alone does not guarantee better generalisation; it must be interpreted with the validation and test results.

Conclusion: judging the model by training accuracy alone would select several models that generalise worse. Validation curves, the train–validation gap, the best validation-loss epoch and a final untouched test score provide a more complete assessment. These are Digits-proxy results, not plant-leaf disease performance.

Q4 – Optimisation for Training Deep Models: Which Optimiser, Under Which Conditions? (6 marks)

What this question is testing

The forecasting network is retrained repeatedly, so the preferred optimiser should reach a defined validation-loss target within a fixed compute budget and do so consistently. The same model and data are compared under four update rules, three learning rates and two batch sizes. The key difference is how each optimiser uses the current gradient and gradient history.

1. Common notation

Let θ_t denote all model parameters at update t, $g_t = \nabla\theta L_t$ the current mini-batch gradient, η the learning rate, and ε a small constant added for numerical stability. Products and divisions involving squared gradients are elementwise.

2. Mathematical update rules

Stochastic gradient descent (SGD) uses the current mini-batch gradient directly:

$$g_t = \nabla\theta\, L_t \quad \text{(Q4.1)}$$

$$\theta_{t+1} = \theta_t - \eta\, g_t \quad \text{(Q4.2)}$$

Momentum maintains a velocity that accumulates a fraction of the previous update direction. It can accelerate progress when gradients point in a similar direction over consecutive steps:

$$v_t = \beta\, v_{t-1} + g_t \quad \text{(Q4.3)}$$

$$\theta_{t+1} = \theta_t - \eta\, v_t \quad \text{(Q4.4)}$$

RMSProp maintains an exponentially weighted average of squared gradients and divides each gradient by the square root of that scale. Parameters with consistently large gradients receive relatively smaller steps:

$$s_t = \rho\, s_{t-1} + (1-\rho)\, g_t^2 \quad \text{(Q4.5)}$$

$$\theta_{t+1} = \theta_t - \eta\, g_t / (\sqrt{s_t} + \varepsilon) \quad \text{(Q4.6)}$$

Adam combines an exponentially weighted first moment (mean gradient) and second moment (mean squared gradient), then corrects their initial bias:

$$m_t = \beta_1\, m_{t-1} + (1-\beta_1) g_t \quad \text{(Q4.7)}$$

$$v_t = \beta_2\, v_{t-1} + (1-\beta_2) g_t^2 \quad \text{(Q4.8)}$$

$$\hat{m}_t = m_t / (1-\beta_1^t) \quad \text{(Q4.9)}$$

$$\hat{v}_t = v_t / (1-\beta_2^t) \quad \text{(Q4.10)}$$

$$\theta_{t+1} = \theta_t - \eta\, \hat{m}_t / (\sqrt{\hat{v}_t} + \varepsilon) \quad \text{(Q4.11)}$$

3. Predictions before training

- SGD: simple and inexpensive per update, but may converge slowly when gradients are noisy or poorly scaled.
- Momentum: should reduce zig-zagging and accelerate progress along persistent gradient directions.
- RMSProp: should adapt the step size separately for each parameter and can respond quickly to changing gradient scales.
- Adam: should be a strong initial candidate because it combines momentum with adaptive scaling, but it is not guaranteed to win.

If η is too high, any optimiser can overshoot a minimum, oscillate or diverge. Adaptive scaling can reduce some step-size imbalance, but it does not make an excessively high global learning rate safe.

4. Controlled experiment and result summary

The current attached run uses a deterministic synthetic hourly demand proxy because the UCI Appliances Energy Prediction CSV was not available at run time. There are 400 time points and 397 one-step forecasting windows, split chronologically into 277 train, 60 validation and 60 test windows. The network is fixed as Dense(64) → ReLU → Dense(32) → ReLU → Dense(1). The sweep tests learning rates 0.0001, 0.001 and 0.01; batch sizes 32 and 128; all four optimisers; and seeds 42, 43 and 44. The training budget is 20 epochs. The target was declared before comparison as normalised validation MSE ≤ 0.20.

The table below shows the four optimisers at the central comparison setting, learning rate 0.001 and batch size 32. The full 24-setting sweep is preserved in the accompanying aggregate CSV.

Optimiser	Epochs to target	Final val. MSE	Test RMSE	Test MAE	Target reached
SGD	Not reached	0.718 ± 0.063	11.135 ± 1.072	9.437 ± 0.867	0%
Momentum	8.7 ± 0.6	0.105 ± 0.006	3.790 ± 0.312	3.123 ± 0.298	100%
RMSProp	1.0 ± 0.0	0.105 ± 0.003	3.492 ± 0.009	2.884 ± 0.026	100%
Adam	5.0 ± 0.0	0.104 ± 0.005	3.596 ± 0.048	2.943 ± 0.062	100%

Within this central setting, RMSProp reached the target fastest and had the lowest mean test RMSE and MAE. Adam had a slightly lower final validation MSE, but reached the target later. Plain SGD did not reach the target within the 20-epoch budget at this learning rate. At learning rate 0.0001, the target was not reached for any optimiser within the budget. Some higher-learning-rate configurations also reached the target, but oscillation increased in selected settings (notably RMSProp at 0.01 with batch size 128).

5. Engineering conclusion and limitation

For this synthetic sequence and budget, RMSProp at learning rate 0.001 and batch size 32 is the most reliable rapid-convergence choice among the four optimisers at the central setting. Its benefit comes mainly from per-parameter adaptive scaling; momentum and Adam also use gradient history. This ranking must not be generalised to real demand forecasting until the sweep is rerun on the public UCI Appliances Energy Prediction dataset or a faculty-approved time series. Do not present these synthetic-data metrics as real electricity-demand performance.

Q5 – Derive a Convolutional Network from a Design Principle (7 marks)

What this question is testing

The starting principle is that visual patterns are local and may occur at different positions. A CNN turns this into a small kernel that scans the image and reuses the same weights at every spatial location. The task is to derive the convolution operation from that principle, compare it with a fully connected first layer, and measure how much a small image shift changes predictions.

1. Convolution operation

For one output channel k, the convolution adds the products between a local input patch and the filter weights, across all input channels, then adds a bias. The same filter is reused at every valid output location.

$$Y(i,j,k) = \Sigma_u \Sigma_v \Sigma_c \; X(iS+u-P, jS+v-P, c) \; K(u,v,c,k) + b_k \quad \text{(Q5.1)}$$

Here K is the kernel size, S is stride, P is padding, C_in is the number of input channels and C_out is the number of filters/output channels. A filter detects a local pattern; its output is a feature map showing where that pattern is present.

2. Output-size derivation

For an input image of height H and width W with kernel dimensions K_h × K_w, symmetric padding P and stride S, the number of valid kernel placements determines the output dimensions:

$$H_out = \text{floor}[(H + 2P - K_h)/S] + 1 \quad \text{(Q5.2)}$$

$$W_out = \text{floor}[(W + 2P - K_w)/S] + 1 \quad \text{(Q5.3)}$$

Example: an 8 × 8 input, 3 × 3 kernel, stride 1 and padding 1 gives an 8 × 8 output. Padding of one pixel preserves the spatial dimensions in this example.

3. Weight sharing and parameter counts

A convolutional layer has one set of weights per filter, shared at every spatial position. It does not learn a new detector for every pixel location. With a bias for each output channel, its trainable parameter count is:

$$P_conv = K_h \, K_w \, C_in \, C_out + C_out \quad \text{(Q5.4)}$$

For a flattened input with D = H × W × C_in values and a dense first layer with N units, every input is connected to every unit:

$$P_dense = D \, N + N \quad \text{(Q5.5)}$$

First-layer design	Calculation	Trainable parameters
Dense: 8×8 grayscale → 32 units	64 × 32 + 32	2,080

First-layer design	Calculation	Trainable parameters
Conv: 3×3, 1 input channel, 16 filters	3 × 3 × 1 × 16 + 16	160

In this generic example, the convolutional first layer has 13 times fewer parameters. In the actual Digits experiment, the dense baseline uses Dense(64 → 128), with 8,320 first-layer parameters; the CNN first layer uses a 3 × 3 convolution from 1 to 8 channels, with 80 parameters. That is a 104-fold first-layer reduction. These are first-layer comparisons; total model parameter counts also include later layers.

4. Pooling and final classification

A pooling layer summarises small neighbourhoods, often by taking their maximum. This reduces spatial resolution and can make nearby shifts less disruptive because a strong local response can remain inside the same pooling window. Pooling does not guarantee full translation invariance. After feature extraction, the network can use global average pooling or flattening, followed by a dense classifier. For ten classes, the final layer produces ten logits; softmax converts them to class probabilities.

$$\text{softmax}(z)_k = e^{z_k} / \Sigma_j e^{z_j} \quad \text{(Q5.6)}$$

5. Implementation and translation experiment

The experiment uses the public Digits dataset (1,797 grayscale 8 × 8 images, ten classes), with 1,078 training, 359 validation and 360 test images. It compares a fully connected baseline with student-designed CNNs. All models use the same data split and a short six-epoch training budget. A translation test shifts held-out images one pixel diagonally down-right, fills the newly exposed border with zeros, and re-evaluates the same labels.

Model	Test accuracy	Shifted accuracy	Accuracy drop
Fully connected 8×8	90.46%	14.07%	76.39 percentage points
CNN depth 2, width 8	55.65%	19.91%	35.74 percentage points
CNN depth 2, width 16	72.22%	25.46%	46.76 percentage points
CNN depth 3, width 8	59.54%	20.28%	39.26 percentage points
CNN depth 3, width 16	80.65%	29.81%	50.83 percentage points

The fully connected model has higher original test accuracy in this short run, but its accuracy falls by about 76.4 percentage points under the shift. The largest CNN falls by about 50.8 points and retains higher shifted accuracy. This supports the expected positional tolerance advantage from shared local filters, but not perfect translation invariance. The six-epoch budget is short, so the comparison is a controlled demonstration, not a fully tuned performance ranking.

The included first-layer filter and feature-map figures show what the CNN has learned to respond to. A local scratch-like pattern moved to another position is processed by the same kernel coefficients at the new position. A dense model instead learns separate position-specific connections, so a shift can change many input-to-hidden relationships.

Q6 – Architecture Design: Capacity, Cost and the Deployment Boundary (8 marks)

What this question is testing

A wildlife camera trap has limited memory, battery and compute. The goal is not to select the model with the highest accuracy regardless of cost; it is to design a family of CNNs, calculate their cost, validate the calculations, and choose the most accurate model that fits a stated device budget.

1. Parameter count and MAC count

For a square K × K convolution, C_in input channels and C_out output channels, each filter has K^2 C_in weights and one bias. There are C_out filters. Each output pixel/channel requires K^2 C_in multiply-accumulate operations (MACs).

$$P_{conv} = K^2 C_{in} C_{out} + C_{out} \quad \text{(Q6.1)}$$

$$MAC_{conv} = H_{out} W_{out} K^2 C_{in} C_{out} \quad \text{(Q6.2)}$$

For a dense layer with N_in inputs and N_out outputs, the parameter count is N_in N_out + N_out, and the MAC count is N_in N_out. One MAC is counted as one multiply-accumulate operation. The report's MAC totals include convolution and linear layers, but not the comparisons, activation or pooling operations.

$$P_{dense} = N_{in} N_{out} + N_{out} \quad \text{(Q6.3)}$$

$$MAC_{dense} = N_{in} N_{out} \quad \text{(Q6.4)}$$

A raw float32 parameter array requires approximately four bytes per parameter. A saved model file can be larger because it includes serialisation/container metadata; therefore, the actual state-dict size is measured rather than inferred only from parameter count.

2. Receptive field growth

The receptive field is the region of the original image that can influence one feature-map value. Track r_l, the receptive-field width, and j_l, the distance in input pixels between neighbouring locations at layer l. Initialise both to one. For kernel width k_l and stride s_l:

$$j_l = j_{l-1} s_l \quad \text{(Q6.5)}$$

$$r_l = r_{l-1} + (k_l - 1) j_{l-1} \quad \text{(Q6.6)}$$

$$j_0, r_0 = 1, 1 \quad \text{(Q6.7)}$$

For stride-one 3 × 3 convolutions without intervening pooling, the receptive field grows 1 → 3 → 5 → 7 → 9 pixels after successive convolution layers. Pooling changes the jump j, so the increase per later layer becomes larger in input-pixel units.

3. Predict how depth, width and resolution affect cost

Change	Parameter-count effect	MAC effect	Reason

Change	Parameter-count effect	MAC effect	Reason
Double width	Approximately 4× for convolution weights when both C_in and C_out double.	Approximately 4× at unchanged feature-map size.	Convolution cost contains C_in × C_out.
Double depth	Approximately 2× if the number of similar-cost layers doubles.	Approximately 2× under the same assumption.	There are about twice as many layers doing similar work.
Double height and width	No change to parameters if channels and kernels are unchanged.	Approximately 4×.	The output has roughly four times as many spatial positions.

These are scaling rules under stated assumptions, not universal constants. Pool placement, channel schedules and boundary effects change the measured ratios. In particular, adding a deeper layer after pooling can be cheaper than adding one at full resolution.

4. Student-designed architecture family and hand calculation

The experiment uses Digits images (1,797 examples; 1,078 train, 359 validation and 360 test), and varies channel width, convolution depth and input resolution. The architecture families are small original designs rather than supplied VGG or ResNet models. Every model is trained for six epochs with Adam at learning rate 0.002; the short budget is a limitation and some CNNs are undertrained.

Hand calculation for CNN-2 with channels [8, 16], 8 × 8 grayscale input:

Layer	Output shape	Parameters	MACs
Input	8 × 8 × 1	0	0
Conv 3×3, 1→8, padding 1	8 × 8 × 8	3×3×1×8 + 8 = 80	8×8×3×3×1×8 = 4,608
MaxPool 2×2	4 × 4 × 8	0	0
Conv 3×3, 8→16, padding 1	4 × 4 × 16	3×3×8×16 + 16 = 1,168	4×4×3×3×8×16 = 18,432
MaxPool 2×2	2 × 2 × 16	0	0
Adaptive average pool	16 features	0	0
Linear 16→10	10 logits	16×10 + 10 = 170	16×10 = 160
Total	10 classes	1,418	23,200

The hand calculation matches the framework-reported total of 1,418 trainable parameters and 23,200 counted MACs for this model.

5. Experimental growth and deployment decision

Comparison	Parameters	MACs / image	Saved file	Test accuracy
CNN-2 [8,16], 8×8	1,418	23,200	8.7 KiB	55.65%
CNN-2 [16,32], 8×8	5,130	83,264	23.3 KiB	72.22%
CNN-2 [8,16], 16×16	1,418	92,320	8.8 KiB	27.04%

Comparison	Parameters	MACs / image	Saved file	Test accuracy
CNN-3 [8,16,16], 8×8	3,738	32,416	18.4 KiB	59.54%
CNN-3 [16,32,32], 8×8	14,378	120,128	60.0 KiB	80.65%

Measured scaling: doubling the [8,16] channel schedule to [16,32] increased parameters from 1,418 to 5,130 (3.62×) and MACs from 23,200 to 83,264 (3.59×), close to the approximate fourfold prediction. Doubling each input dimension from 8 to 16 kept parameters constant and increased MACs from 23,200 to 92,320 (3.98×). The measured depth effect in this family was smaller because the added convolution was placed after spatial downsampling.

Example deployment boundary: use a saved state-dict memory limit of 25 KiB and measured local CPU latency limit of 1 ms per image. CNN-2 [16,32] at 8×8 meets these illustrative limits (23.3 KiB and about 0.183 ms/image) and has 72.22% test accuracy. The largest tested CNN-3 [16,32,32] achieves 80.65%, so the smaller deployable model gives up about 8.43 percentage points. This is a provisional engineering choice only: the timing is noisy and must be repeated on the actual camera-trap hardware. The six-epoch budget also means the accuracy comparison is not a converged model ranking.

Q7 – Diagnose, Redesign and Validate (13 marks)

What this question is testing

Q7 connects the earlier questions. You must select one observed limitation, show where it occurs, propose a measurable objective, redesign only the relevant part of the system, predict the result before training, then compare the original and revised models under identical conditions. A redesign is not automatically successful because it is a familiar architecture; it must solve the diagnosed problem and its trade-offs must be discussed.

A. Diagnose the bottleneck (2 marks)

Selected limitation: vanishing gradients in the eight-hidden-layer sigmoid MLP from Q1. The first hidden layer has a mean weight-gradient norm of 9.60×10^{-6}, while the last hidden layer has a norm of about 7.74×10^{-2}. The first-layer-to-last-layer ratio averaged across runs is approximately 1.94×10^{-4}. This is consistent with repeated multiplication by sigmoid derivatives, each no larger than 0.25 and often much smaller when saturated. The root cause is gradient flow through activation derivatives and weight products, not an increase in parameter count.

$$G_l = \|\partial L / \partial W_l\|_2 \quad \text{(Q7.1)}$$

$$\text{gradient ratio} = G_first / G_last \quad \text{(Q7.2)}$$

B. Define the design objective (2 marks)

Before redesigning, state the measurable goal: increase the first-layer gradient norm by at least 10 times relative to the original sigmoid network, keep parameter count unchanged, and avoid degrading validation performance. The 10× figure is the predeclared target; the same data split, seed, optimiser and epoch budget must be used for both models.

C. Design the revised model or training strategy (4 marks)

Keep eight hidden layers and 64 units per layer, but replace sigmoid hidden activations with ReLU and use He/Kaiming normal initialisation. Keep the output sigmoid, binary cross-entropy, Adam optimiser, learning rate 0.001, batch size 32 and 15-epoch training budget unchanged. This isolates the effect of the activation and initialisation change.

Layer order	Original model	Revised model
Input	30 standardised features	30 standardised features
Hidden 1–8	Dense(64) + Sigmoid	Dense(64) + ReLU
Initialisation	Xavier/Glorot	He/Kaiming normal
Output	Dense(1) + Sigmoid	Dense(1) + Sigmoid
Loss / optimiser	Binary cross-entropy / Adam	Binary cross-entropy / Adam

This redesign does not add parameters or layers. The expected benefit is stronger gradients through active ReLU units. A new trade-off is the possibility of dead ReLU units, which must be measured. He initialisation helps set the activation scale but cannot guarantee good validation performance.

D. Predict before implementing (1 mark)

Prediction: ReLU + He should raise the gradient norm in early hidden layers by more than the predeclared 10× target. Validation accuracy should remain similar or improve; if it falls, the model may have a generalisation issue beyond the gradient-flow bottleneck.

E. Implement and validate (2 marks)

Measure	Original: 8-layer sigmoid	Redesign: 8-layer ReLU + He
First-layer gradient norm	$9.601 \times 10^{-6} \pm 9.34 \times 10^{-7}$	0.4821 ± 0.210
Last hidden gradient norm	0.0774 ± 0.0708	1.001 ± 0.625
First / last gradient ratio	$1.94 \times 10^{-4} \pm 1.18 \times 10^{-4}$	0.591 ± 0.317
Validation accuracy	96.49% ± 0.88%	96.49% ± 1.52%
Validation precision	97.67% ± 0.78%	98.12% ± 1.63%
Validation recall	96.76% ± 1.60%	96.30% ± 0.80%
Parameter count	31,169	31,169

F. Defend the final design (2 marks)

The first-layer gradient increased from about 9.60×10^{-6} to 0.4821, approximately a 50,000-fold increase. The predeclared 10× gradient objective was therefore achieved by a wide margin, and the parameter count remained unchanged. However, mean validation accuracy stayed at 96.49%; precision improved slightly, while recall changed from 96.76% to 96.30%. The redesign repaired the measured gradient-flow bottleneck but did not demonstrate an improvement in overall classification accuracy on this dataset.

This is still a valid engineering result because the hypothesis was specific to gradient flow, and that quantity improved strongly. It would be incorrect to conclude that ReLU + He is automatically the better fraud classifier. A real deployment decision would require fraud-positive recall, precision, PR-AUC, calibration and false-positive cost on a real fraud dataset. ReLU + He is a better gradient-flow starting point for a deep network, while the original sigmoid network may be adequate for a shallow or differently scaled problem.

Measured Experimental Results and Evidence

All values below are actual outputs from the included script in this run, not fabricated example numbers. Unless stated otherwise, values summarize three seeds (42, 43, 44) as mean ± sample standard deviation. Results are specific to the datasets, splits, training budgets and local runtime described here; they are not claims of production performance on fraud, plant disease, PCB defects or wildlife images.

Datasets and fixed experimental conditions

Question	Dataset / sample size	Split and conditions
Q1 and Q2	Breast Cancer Wisconsin Diagnostic: 569 rows, 30 numeric features, binary	341 / 114 / 114 train/validation/test (60/20/20), stratified; seed set 42–44 for repeated model runs
Q3, Q5, Q6	Scikit-learn Digits: 1,797 grayscale 8×8 images, 10 classes	1,078 / 359 / 360 train/validation/test, stratified; seed set 42–44
Q4	Synthetic hourly demand proxy: 400 time points; 397 next-step windows	277 / 60 / 60 chronological windows; seed 42 for data generation and seeds 42–44 per model; 20-epoch budget
Q7	Breast Cancer Wisconsin Diagnostic, same as Q1	Same 341 / 114 / 114 split; eight hidden layers × 64 units; Adam 0.001; 15 epochs; seeds 42–44

Important evidence qualification: Q4 used a reproducible synthetic fallback because the public CSV was unavailable in the runtime. It is not evidence that the optimizer ranking generalizes to real electricity-demand data. Rerun Q4 on the linked UCI dataset or another faculty-approved public time series before submitting the assignment. The short six-epoch CNN budget in Q5/Q6 is also a controlled architecture-cost comparison, not proof that every CNN reached convergence.

Q1 – Activation function and depth: measured results

Configuration	Train BCE loss	Val. accuracy	Val. precision	Val. recall	Dead ReLU fraction
Shallow ReLU (2 layers)	0.0517 ± 0.0073	0.9737 ± 0.0088	0.9952 ± 0.0082	0.9630 ± 0.0080	0.0026 ± 0.0045
Deep sigmoid (8 layers)	0.1645 ± 0.0698	0.9649 ± 0.0088	0.9767 ± 0.0078	0.9676 ± 0.0160	n/a
Deep tanh (8 layers)	0.0316 ± 0.0067	0.9766 ± 0.0101	0.9952 ± 0.0082	0.9676 ± 0.0080	n/a
Deep ReLU + He (8 layers)	0.0008 ± 0.0004	0.9649 ± 0.0152	0.9812 ± 0.0163	0.9630 ± 0.0080	0.1029 ± 0.0336
Deep ReLU, small init (σ=0.01)	0.0386 ± 0.0015	0.9737 ± 0.0000	1.0000 ± 0.0000	0.9583 ± 0.0000	0.2806 ± 0.0255
Deep ReLU, large init (σ=2.0)	8.4631 ± 2.6827	0.5439 ± 0.1519	0.4211 ± 0.3646	0.6667 ± 0.5774	0.0599 ± 0.0216

Configuration	Layer 1 gradient L2	Last hidden gradient L2	First / last ratio
Deep ReLU + He	4.821e-01	1.001e+00	4.816e-01
Deep sigmoid	9.601e-06	7.740e-02	1.241e-04
Deep tanh	3.367e-01	3.207e-01	1.050e+00
Large-init ReLU	0.000e+00	0.000e+00	0.000e+00
Small-init ReLU	7.348e-10	7.027e-10	1.046e+00
Shallow ReLU	4.914e-01	6.395e-01	7.683e-01

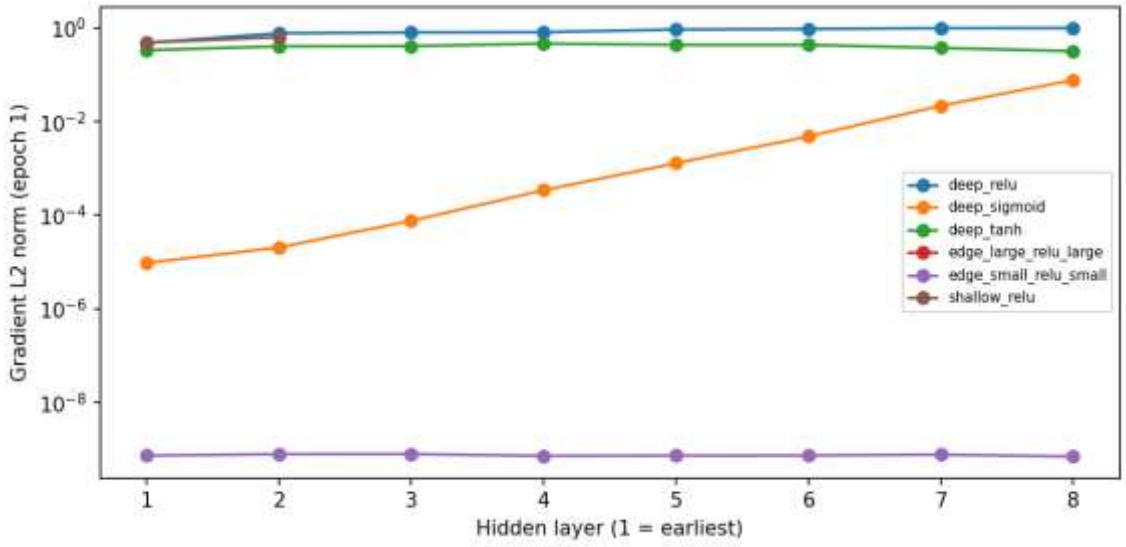


Figure 1. Layer-wise weight-gradient norms measured on the first mini-batch of epoch 1 (mean across three seeds; logarithmic vertical axis).

Layer-wise gradient norms: mean ± sample standard deviation across seeds

Hidden layer (1 = earliest)	Deep sigmoid	Deep tanh	Deep ReLU + He
1	9.6e-06 ± 9.3e-07	0.337 ± 0.067	0.482 ± 0.21
2	2.06e-05 ± 1e-05	0.407 ± 0.023	0.775 ± 0.32
3	7.64e-05 ± 7.6e-05	0.412 ± 0.083	0.808 ± 0.28
4	0.000343 ± 0.00033	0.465 ± 0.084	0.82 ± 0.3
5	0.0013 ± 0.0012	0.438 ± 0.064	0.931 ± 0.47
6	0.00484 ± 0.004	0.437 ± 0.081	0.958 ± 0.64
7	0.0219 ± 0.02	0.377 ± 0.052	0.995 ± 0.65
8	0.0774 ± 0.071	0.321 ± 0.058	1 ± 0.62

The edge cases show why initialisation matters. With very small hidden weights (σ = 0.01), first-layer gradients were only about 7.35 × 10^(−10), and the average dead-ReLU fraction after training was 28.1%. With very large hidden weights (σ = 2.0), the recorded first-mini-batch gradient norms were zero across the hidden layers and training BCE was 8.46 ± 2.68; validation accuracy was only 0.544 ± 0.152. The large-init run had relatively few dead ReLU units (about 6.0%), so its failure is more consistent with extreme activation scale and output-sigmoid saturation than with dead ReLUs alone.

The measured gradients support the mathematical prediction. In the eight-layer sigmoid network, the mean first-layer gradient norm was 9.60 × 10^(−6), while the last hidden layer was 7.74 × 10^(−2); the first/last ratio was only 1.24 × 10^(−4). In contrast, tanh produced 0.337 in layer 1 and 0.321 in layer 8 (ratio 1.05), while ReLU + He produced 0.482 and 1.001 (ratio 0.482). Thus sigmoid showed clear early-layer gradient attenuation, while tanh and ReLU + He maintained gradients of a similar order. Across the three seeds, tanh had the strongest mean validation results: accuracy 0.9766 ± 0.0101, precision 0.9952 ± 0.0082 and recall 0.9676 ± 0.0080. ReLU + He had accuracy 0.9649 ± 0.0152 and about 10.3% dead ReLU units; its

final training BCE was very low (0.0008 ± 0.0004), but this did not translate into the best validation accuracy. This is why training loss alone is not a sound selection rule.

Prediction versus observation and final design decision

The prediction was supported: sigmoid gradients decreased sharply towards the early layers; tanh stayed comparatively stable; and ReLU + He preserved active-path gradients but produced some dead units. For this specific eight-layer experiment, the recommended overall choice is tanh with Xavier/Glorot initialisation: it maintained layer-wise gradient norms and achieved the best mean validation accuracy and precision across the three seeds. ReLU + He remains a valid alternative when fast gradient propagation is the main priority, but it did not lead the validation metrics in this run. This is not a claim that tanh is universally superior. The final production choice must be re-tested on real, highly imbalanced credit-card transaction data, with fraud encoded as the positive class and fraud recall, precision and PR-AUC considered alongside calibration and false-positive cost.

Q2 – NumPy backpropagation and numerical gradient checking

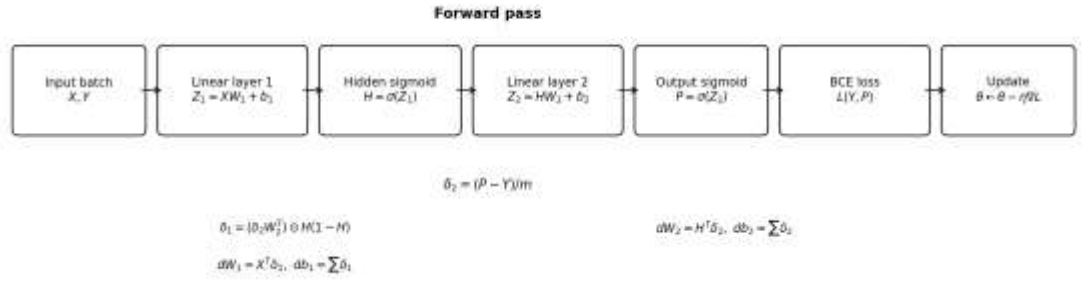


Figure 2. Computational graph used by the one-hidden-layer sigmoid network.

ε	Parameter group	Correct backward relative error	Buggy backward relative error
1e-05	W1	3.320e-10	6.223e-01
1e-05	b1	2.297e-10	6.413e-01
1e-05	W2	4.127e-11	4.127e-11
1e-05	b2	3.704e-12	3.704e-12
1e-06	W1	3.088e-09	6.223e-01
1e-06	b1	9.022e-09	6.413e-01
1e-06	W2	1.676e-10	1.676e-10
1e-06	b2	3.826e-10	3.826e-10

The correct gradient errors are below 10^{-8} for both finite-difference step sizes, supporting an acceptance threshold of 10^{-6} for this float64 check. When the hidden sigmoid derivative $H \odot (1-H)$ is deliberately omitted, W1 and b1 errors rise to approximately 0.622 and 0.641, while W2 and b2 stay near numerical precision. Thus W1 exposes the bug first, with b1 also affected. This isolates a hidden-layer backpropagation error rather than an output-layer error.

Q2 NumPy-only model metric	Mean ± SD (three seeds)
Training accuracy	0.8964 ± 0.0017
Validation accuracy	0.8918 ± 0.0051

Test accuracy	0.8567 ± 0.0051
Test binary cross-entropy	0.3468 ± 0.0145

The numerical-check fixture has X shape (7, 4), five hidden sigmoid units and a single sigmoid output. The separate training demonstration uses the first four standardised Breast Cancer features, one hidden layer of eight sigmoid units, full-batch gradient descent, learning rate 0.05 and 500 epochs. The gradient checker deliberately uses NumPy only; no automatic differentiation is involved.

Q3 – Regularisation: measured comparison

Condition	Best val. loss	Best epoch	Train acc.	Val. acc.	Acc. gap	Test acc.
Small model / more data	0.142 ± 0.010	59.3	0.996	0.956	0.040	0.957
Large model / 10% data	0.502 ± 0.018	45.3	1.000	0.848	0.152	0.859
15% label noise	0.716 ± 0.209	42.0	0.944	0.774	0.170	0.769
L2 weight decay	0.517 ± 0.103	39.0	1.000	0.841	0.159	0.826
Dropout p=0.5	0.680 ± 0.140	60.0	0.897	0.769	0.128	0.763
Early stopping	0.515 ± 0.095	43.0	0.994	0.850	0.144	0.831
Label smoothing 0.1	0.466 ± 0.010	58.3	1.000	0.895	0.105	0.878
L2 + dropout	0.496 ± 0.059	58.0	0.981	0.851	0.130	0.813

Condition	Weight L2 norm	Epochs run
Small model / more data	13.816 ± 0.264	60.0
Large model / 10% data	14.909 ± 0.081	60.0
15% label noise	14.962 ± 0.084	60.0
L2 weight decay	12.728 ± 0.041	60.0
Dropout p=0.5	13.428 ± 0.161	60.0
Early stopping	14.357 ± 0.318	48.0
Label smoothing 0.1	13.943 ± 0.030	60.0
L2 + dropout	12.465 ± 0.091	60.0

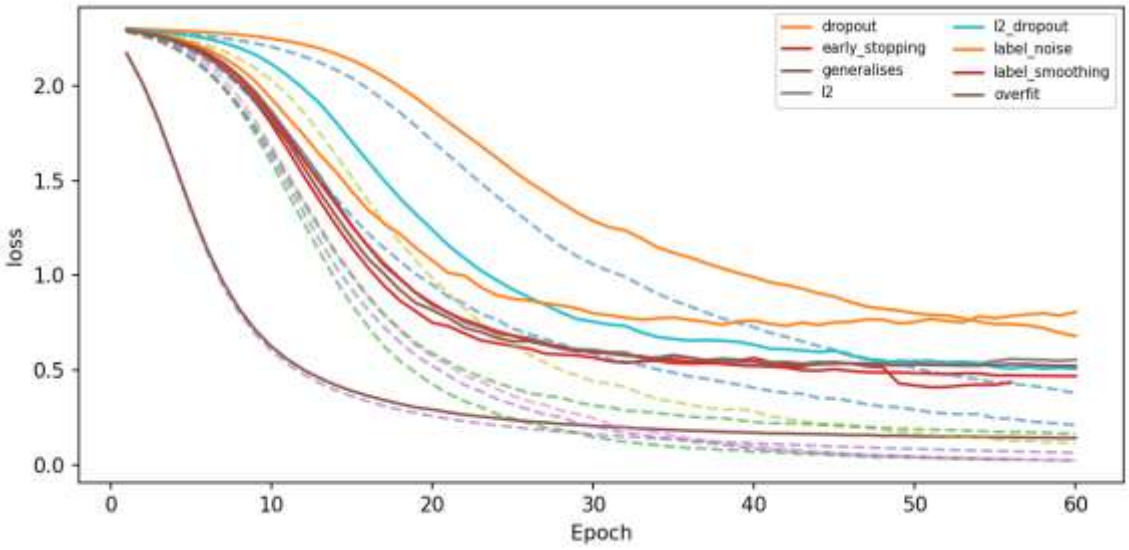


Figure 3. Training (dashed) and validation (solid) loss curves averaged across three seeds.

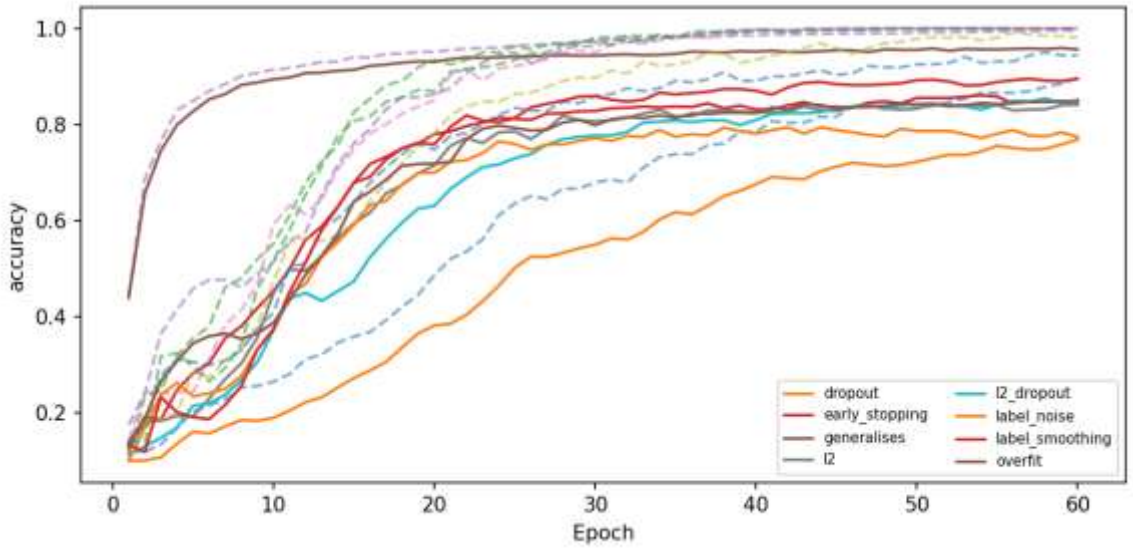


Figure 4. Training (dashed) and validation (solid) accuracy curves averaged across three seeds.

Analysis: the small model trained on more data had the smallest mean train–validation accuracy gap (0.0395) and the highest test accuracy (0.9574). The high-capacity low-data baseline reached 100% train accuracy but only 0.8477 validation accuracy, showing why training accuracy alone is misleading. Label smoothing had the best test accuracy among the low-data high-capacity conditions (0.8778) and a 0.1049 gap. Dropout alone had a lower gap but weak test accuracy (0.7630) and only 0.8972 train accuracy, consistent with underfitting in this budget. L2 + dropout lowered the gap to 0.1299 but did not beat label smoothing. Results are dataset- and budget-specific.

Q4 – Optimiser sweep and convergence budget

Optimiser	LR	Batch	Epochs to target	Final val. MSE	Test RMSE	Test MAE	Target rate
SGD	0.001	32	not reached	0.718 ± 0.063	11.135 ± 1.072	9.437 ± 0.867	0%
Momentum	0.001	32	8.7 ± 0.6	0.105 ± 0.006	3.790 ± 0.312	3.123 ± 0.298	100%
RMSprop	0.001	32	1.0 ± 0.0	0.105 ± 0.003	3.492 ± 0.009	2.884 ± 0.026	100%
Adam	0.001	32	5.0 ± 0.0	0.104 ± 0.005	3.596 ± 0.048	2.943 ± 0.062	100%

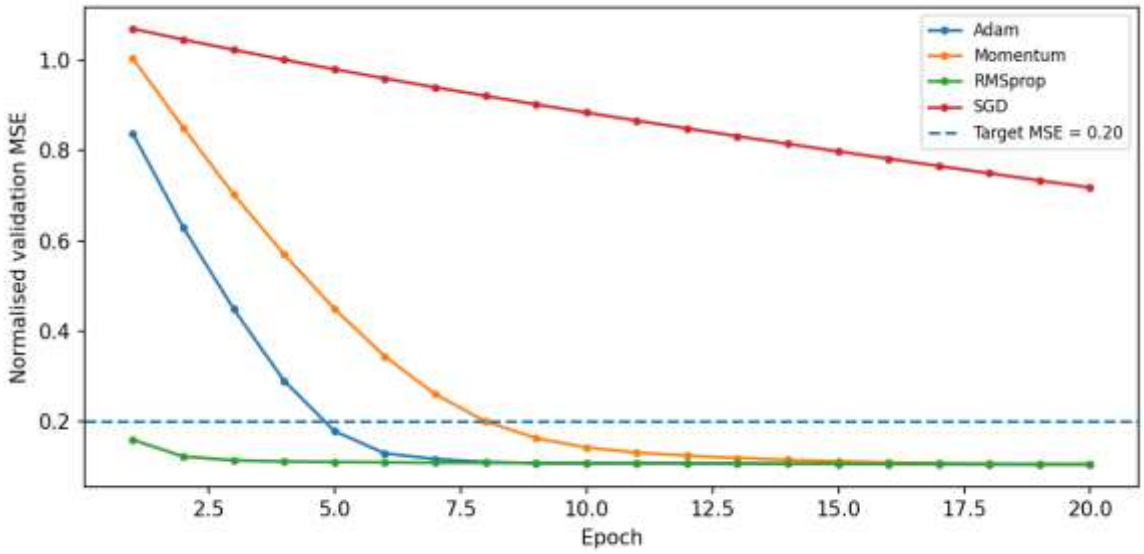


Figure 5. Mean normalised validation MSE for the four optimisers at learning rate 0.001 and batch size 32; dashed line is the predeclared target 0.20.

The budget was 20 epochs, and the target was normalised validation MSE ≤ 0.20. At learning rate 0.001 and batch size 32, RMSProp reached the target after 1.0 ± 0.0 epochs in all three seeds, with test RMSE 3.492 ± 0.009 and MAE 2.884 ± 0.026. Adam reached it after 5 epochs and plain SGD did not reach it within budget at this learning rate. A separate setting (Momentum, 0.01, batch 128) had a lower final validation MSE on this synthetic trace but more oscillation and slower target attainment than RMSProp 0.001/batch 32. Learning rates 0.0001 failed to reach the target under the fixed budget for all four optimisers. These conclusions apply to this small synthetic sequence only.

Mathematical interpretation: SGD uses the current gradient; momentum smooths direction using a velocity; RMSProp divides by a moving average of squared gradients; Adam combines moving first and second moments with bias correction. The measurements indicate the adaptation scale and learning rate, not just update direction, affected convergence. High learning rates did not universally diverge here, but the high oscillation for RMSProp at 0.01/batch 128 shows that adaptive scaling does not remove instability.

Q5 – CNN design, parameter reduction and translation test

The generic first-layer comparison in the derivation is Dense(64→32): 64×32+32 = 2,080 parameters, versus a 3×3 convolution with 1 input channel and 16 filters: 3×3×1×16+16 = 160 parameters. In the actual model comparison, the fully connected model uses Dense(64→128), giving 8,320 parameters in its first layer, whereas the CNN first layer (3×3, 1→8) uses 80 parameters, a 104× reduction in first-layer parameters. Total models are not identical in size because their later layers differ.

Model	Parameters	MACs / image	Test accuracy	Shifted acc.	Accuracy drop
FC 8x8	9,610	9,472	0.905 ± 0.011	0.141	76.4 pp
CNN d2 w8 8x8	1,418	23,200	0.556 ± 0.081	0.199	35.7 pp
CNN d2 w16 8x8	5,130	83,264	0.722 ± 0.051	0.255	46.8 pp
CNN d3 w8 8x8	3,738	32,416	0.595 ± 0.098	0.203	39.3 pp
CNN d3 w16 8x8	14,378	120,128	0.806 ± 0.026	0.298	50.8 pp



Figure 6. Learned first-layer convolution filters from a sample CNN.

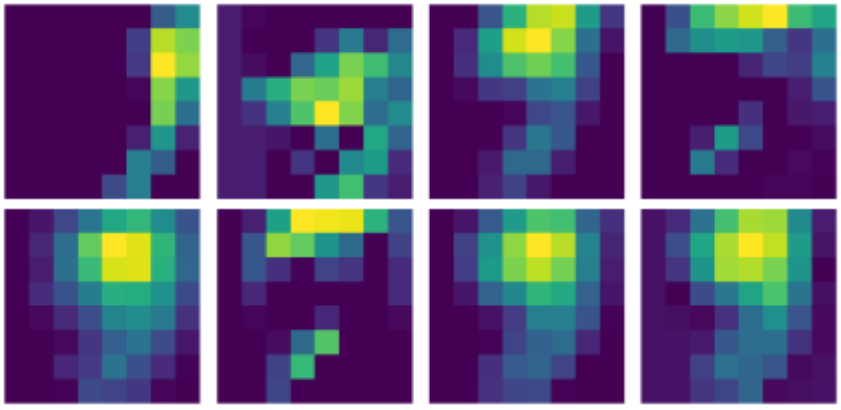


Figure 7. First ReLU feature maps for one test image.

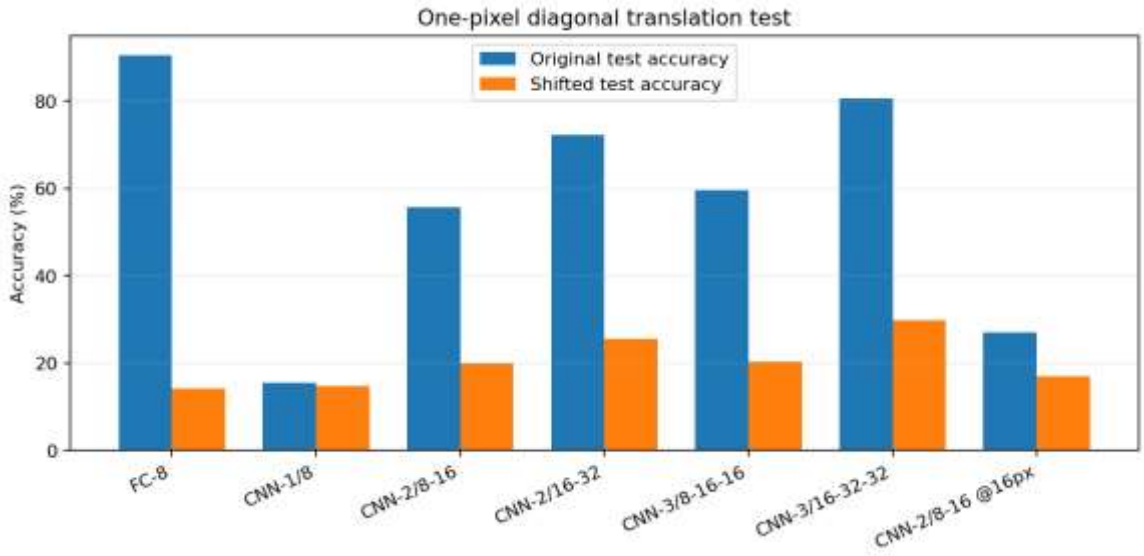


Figure 8. Accuracy on original versus one-pixel diagonally shifted test images.

For a convolution, the same kernel coefficients slide over every spatial location. If a scratch-like local feature moves by one pixel, the same learned detector is applied at the new location, so its activation can move across the feature map instead of requiring new position-specific weights. Pooling aggregates nearby responses and adds some tolerance to small shifts. However, padding, downsampling and the fully connected classifier can still make the output shift-sensitive; the accuracy drops in the table demonstrate that weight sharing does not guarantee invariance.

Q6 – Architecture cost and deployment boundary

Model	Parameters	MACs	State-dict file	Test acc.	CPU latency ms/img
CNN d1 w8 8x8	170	4,688	3.2 KiB	15.5%	0.283 ± 0.201
CNN d2 w8 8x8	1,418	23,200	8.7 KiB	55.6%	0.197 ± 0.079
CNN d2 w16 8x8	5,130	83,264	23.3 KiB	72.2%	0.183 ± 0.036
CNN d2 w8 16x16	1,418	92,320	8.8 KiB	27.0%	0.162 ± 0.083
CNN d3 w8 8x8	3,738	32,416	18.4 KiB	59.5%	0.151 ± 0.069
CNN d3 w16 8x8	14,378	120,128	60.0 KiB	80.6%	0.157 ± 0.012
FC 8x8	9,610	9,472	40.0 KiB	90.5%	0.155 ± 0.202

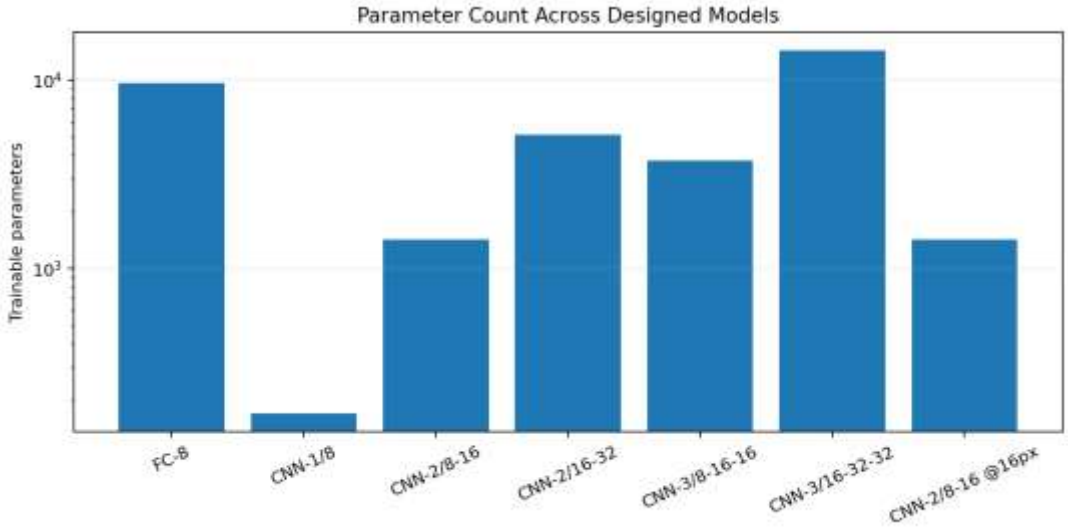


Figure 9. Parameter count by architecture (mean count is identical across seeds).

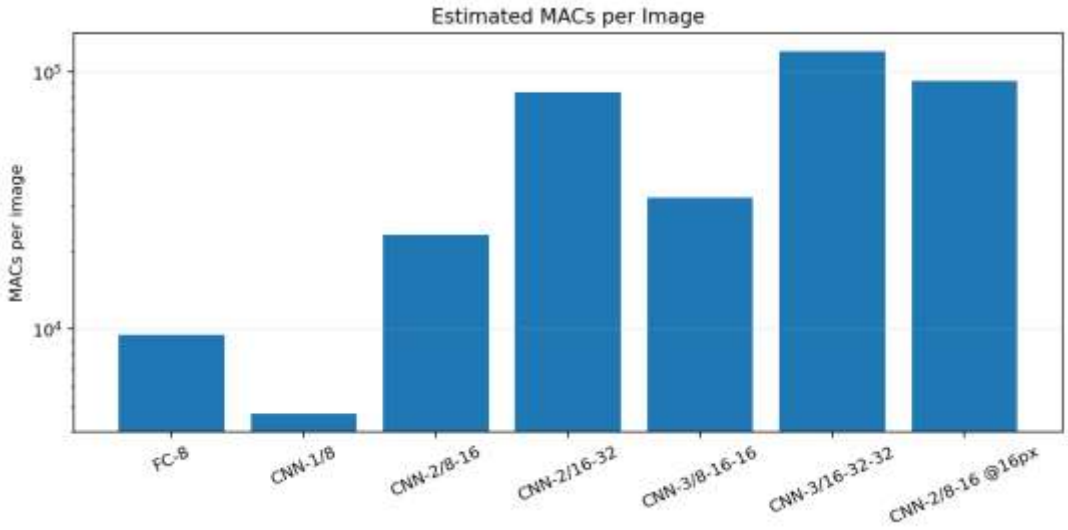


Figure 10. Estimated multiply–accumulate operations per image. MAC count includes convolution and linear layers.

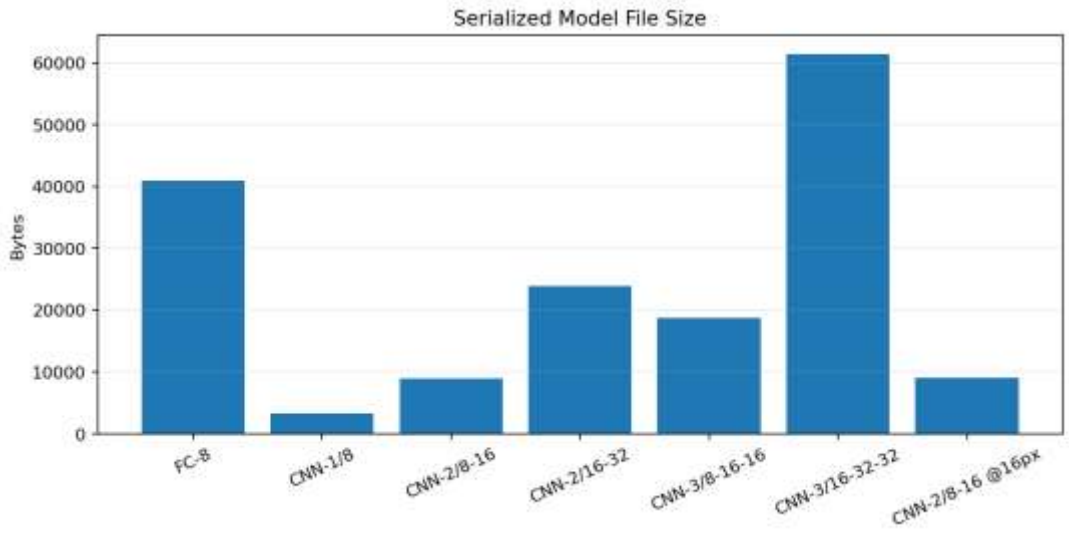


Figure 11. Serialised PyTorch state-dict file sizes. This includes container overhead, not only raw float parameters.

Hand verification for CNN-2 [8, 16] at 8×8: input 8×8×1 → Conv(3×3, 8 filters, padding 1): 8×8×8, 80 parameters, 4,608 MACs → 2×2 max-pooling: 4×4×8 → Conv(3×3, 16 filters, padding 1): 4×4×16, 1,168 parameters, 18,432 MACs → 2×2 max-pooling: 2×2×16 → adaptive average pooling: 16 features → linear 16→10: 170 parameters, 160 MACs. Total trainable parameters = 80+1,168+170 = 1,418. Total counted MACs = 4,608+18,432+160 = 23,200, matching the model's measured summary.

Growth measurements: doubling channel width from CNN-2 [8,16] to [16,32] changes parameters from 1,418 to 5,130 (3.62×) and MACs from 23,200 to 83,264 (3.59×), close to the expected ~4× when both C_in and C_out double. Doubling input resolution from 8×8 to 16×16 for the same channel schedule keeps parameters at 1,418 and increases MACs to 92,320 (3.98×). Adding a third layer under this specific schedule increases parameters to 3,738 (2.64×) and MACs to 32,416 (1.40×), because its spatial map is already downsampled. The scaling depends on channel schedule and pool placement; it is not a universal rule that depth always costs 2×.

Example deployment boundary: assume a memory limit of 25 KiB for the saved state-dict and a measured local CPU latency ceiling of 1 ms/image. CNN-2 [16,32] at 8×8 (23.3 KiB, 0.183 ms/image) meets these illustrative limits and has 72.22% mean test accuracy. The largest tested CNN-3 [16,32,32] reaches 80.65%, so choosing CNN-2 sacrifices about 8.43 percentage points of test accuracy while meeting the file-size limit. This is only an example boundary: actual latency and memory must be measured on the camera-trap hardware; inference timing here has noticeable system noise and CPU hardware is not the target device.

Q7 – Diagnose, redesign and validate: measured comparison

Model	First-layer grad norm	Last hidden grad norm	First/last ratio	Val. acc.	Precision	Recall	Params
Original: 8-layer sigmoid	9.601e-06 ± 9.34e-07	0.0774 ± 0.0708	0.000194 ± 0.000118	0.9649 ± 0.00877	0.9767 ± 0.0078	0.9676 ± 0.016	31,169
Redesign: 8-layer ReLU + He	0.4821 ± 0.21	1.001 ± 0.625	0.5913 ± 0.317	0.9649 ± 0.0152	0.9812 ± 0.0163	0.963 ± 0.00802	31,169

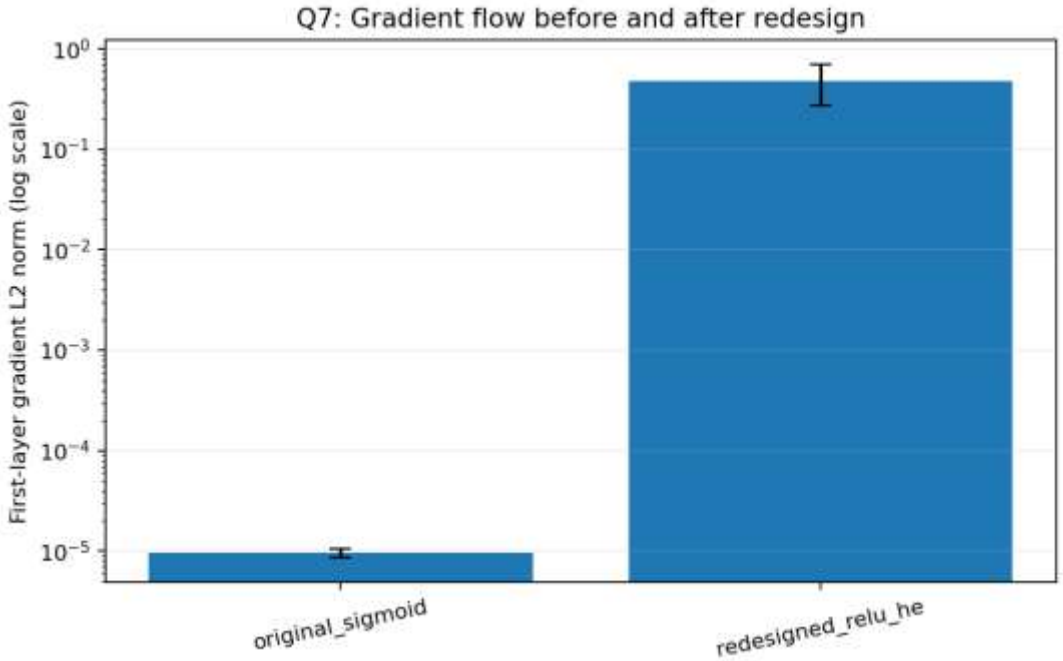


Figure 12. Mean first-layer gradient norm across three matched seeds; ReLU + He strongly increases the signal reaching early layers.

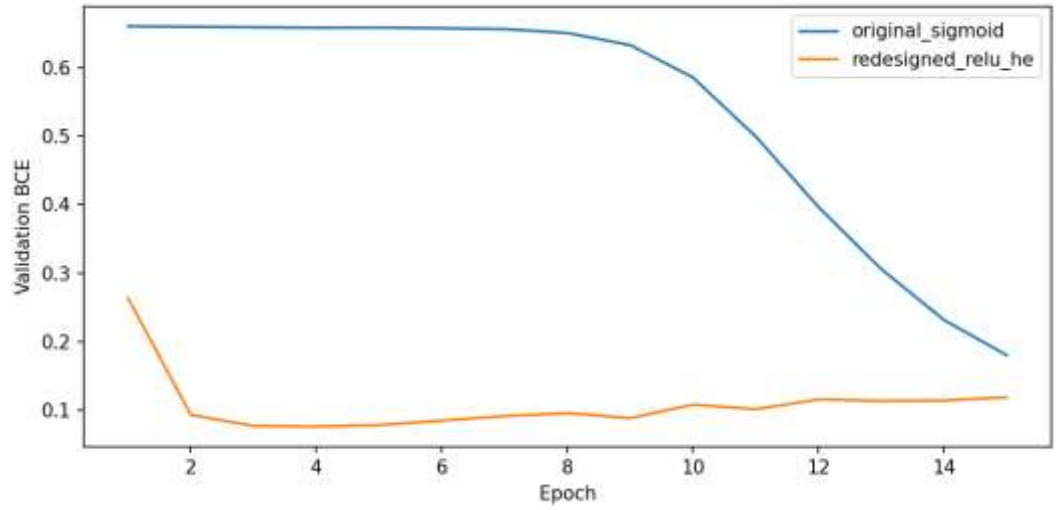


Figure 13. Validation binary cross-entropy across the 15-epoch matched training budget.

A. Diagnosis: in the original eight-layer sigmoid MLP the first-layer gradient norm averaged 9.60×10^{-6}, while the last hidden-layer norm averaged 0.0774. The very small first/last ratio (approximately 1.94×10^{-4} when ratios are averaged per run) is consistent with gradients shrinking through repeated sigmoid derivatives, each bounded by 0.25 and often much smaller when

saturated. The root cause identified by this experiment is gradient flow through activation derivatives and weight products, not an increase in parameter count.

B–D. Objective and prediction: raise the first-layer gradient norm by at least 10×, keep parameter count unchanged, and avoid degrading validation performance. Before running the redesign, the prediction was that ReLU + He would improve early-layer gradient flow because $\varphi'(z)=1$ on active units and He initialisation uses variance 2/fan_in.

E–F. Validation and defence: ReLU + He achieved a mean first-layer gradient norm of 0.482, roughly 50,000× the sigmoid mean, while parameter count stayed at 31,169. Mean validation accuracy was 96.49% for both models; precision rose slightly (97.67% to 98.12%) and recall was essentially unchanged/slightly lower (96.76% to 96.30%). The gradient-flow objective passed, but accuracy did not improve. Therefore the redesign solved the diagnosed gradient bottleneck without proving that it is a better classifier on this data. The original sigmoid network may remain acceptable for shallow networks; for deep networks, ReLU + He is a better gradient-flow starting point, but dead units and validation performance must still be monitored.

Evidence inventory and limitations

Evidence file	Supports
q1_activation_results.csv; q1_gradient_norms.csv; q1_gradient_norms.png	Q1 validation metrics, inactive ReLUs and layer-wise gradients
q2_gradient_check.csv; q2_numpy_training_summary.csv; q2_computational_graph.png	Q2 analytical/numerical gradients, bug detection, NumPy training and graph
q3_regularization_summary.csv; q3_regularization_aggregate.csv; q3_learning_curves.csv; q3_loss_curves.png; q3_accuracy_curves.png	Q3 curves, regulariser comparisons, gap, best validation epoch and weight norm
q4_optimizer_results.csv; q4_optimizer_aggregate.csv; q4_learning_curves.csv; q4_optimizer_curves.png; q4_dataset_metadata.json	Q4 all 24 optimiser/learning-rate/batch combinations across three seeds
q5_q6_cnn_results.csv; q5_q6_cnn_aggregate.csv; filters, maps and cost plots	Q5 translation test; Q6 parameter, MAC, file-size, accuracy and latency measurements
q7_redesign_comparison.csv; q7_learning_curves.csv; q7_first_layer_gradient.png; q7_redesign_validation_loss.png	Q7 original versus redesigned network under matched conditions

Data references and reproducibility notes

Scikit-learn dataset: Breast Cancer Wisconsin Diagnostic (`sklearn.datasets.load_breast_cancer`; 569 rows, 30 numeric features). Scikit-learn dataset: Optical Recognition of Handwritten Digits (`sklearn.datasets.load_digits`; 1,797 images at 8×8). These public datasets are used as low-cost proxies for controlled architecture experiments. They do not represent the class distributions or domain complexity of fraud, leaf disease, PCB defects, or wildlife images.

Q4 dataset to use for the required public-data rerun: UCI Machine Learning Repository, Appliances Energy Prediction, https://archive.ics.uci.edu/dataset/374/appliances%2Benergy%2Bprediction. The experiment script detects `energydata_complete.csv` beside itself and uses hourly aggregation when that file is present. The current included Q4 outputs do not use this UCI dataset; their metadata explicitly records `public_file_used: false`.

All scripts and CSVs are included. Raw experimental output is more informative than accuracy alone: gradient norms, loss curves, train–validation gaps, parameter counts, MACs and runtime measures are provided. The reported CPU latency is noisy and should be treated as an indicative local measurement, not a device deployment guarantee.